

PSI Projekt

Sprawozdanie końcowe

Zespół 34

Miłosz Andryszczuk - Lider

Aleksander Paliwoda

Maksymilian Zieliński

1. Cel projektu

Celem było zaprojektowanie i zaimplementowanie protokołu Mini TLS umożliwiającego bezpieczną komunikację pomiędzy klientem a serwerem. System miał symulować uproszczone działanie protokołu TLS, obejmując:

- uzgadnianie wspólnego klucza metodą Diffiego-Hellmana
- utworzenie szyfrowanej i uwierzytelnionej sesji
- negocjację parametrów kryptograficznych
- wymianę zaszyfrowanych wiadomości

Implementacja została wykonana w Pythonie oraz uruchamiana w środowisku Docker.

2. Założenia i funkcjonalności systemu

System składa się z dwóch głównych części:

Serwer

- akceptuje wielu klientów
- pozwala na listowanie aktywnych klientów oraz rozłączenie ich ręcznie
- prowadzi oddzielne instancje protokołu dla każdej sesji
- odbiera i odszyfrowuje zabezpieczone wiadomości

Klient

- łączy się z serwerem na żądanie
- inicjuje handshake
- wysyła zaszyfrowane wiadomości
- może zakończyć sesję

Funkcjonalności protokołu obejmują:

- szyfrowanie XOR
- handshake DH
- ustanowienia klucza sesji
- integralność poprzez HMAC-SHA256
- format wiadomości typu Encrypt-then-MAC
- flagi sterujące (DATA, END_SESSION)
- własny 5-bajtowy nagłówek ramki (typ + długość payloadu)

3. Architektura systemu

server.py:

- uruchamia nasłuch TCP,
- akceptuje połączenia i tworzy osobne wątki dla klientów
- utrzymuje mapę klientów z ich instancjami protokołu,
- przyjmuje i obsługuje wiadomości zgodnie z maszyną stanów
- Implementuje CLI: list, kill, exit

client.py:

- łączy się z serwerem
- inicjuje handshake
- w osobnym wątku odbiera wiadomości serwera
- Implementuje CLI: connect, send, quit

protocol.py:

realizuje mechanizmy kryptograficzne:

- Diffie-Hellman
- wyprowadza klucz sesji (SHA-256)
- szyfrowanie XOR
- HMAC-SHA256
- buduje ramki
- weryfikuje poprawność i integralność wiadomości

consts.py:

- Definicje formatów nagłówków, rozmiarów oraz typów wiadomości

4. Opis protokołu Mini-TLS

4.1 Handshake

Handshake jest inicjowany przez klienta poprzez wysłanie ClientHello:

- **p, g** - parametry DH
- **$A = g^a \bmod p$** - klucz publiczny klienta.

Serwer odpowiada poprzez ServerHello zawierającym:

- **$B = g^b \bmod p$** - swój klucz publiczny

Obie strony obliczają wspólny sekret:

- **$S = B^a \bmod p$ lub $S = A^b \bmod p$**

Następnie generowany jest klucz sesji:

- **`session_key = SHA256(str(S))`**

4.2 Szyfrowanie danych

Wiadomości są szyfrowane uproszczonym szyfrem strumieniowym XOR:

- **`cipher[i] = plaintext[i] XOR session_key[i % len(session_key)]`**

4.3 Integralność: HMAC-SHA256

Dla każdej wiadomości obliczany jest tag integralności:

- **`HMAC(session_key, ciphertext)`**

4.4 Format wiadomości

Wiadomość zewnętrzna:

- OuterHeader: Type (1B) | Length (4B)
- Payload: Ciphertext | HMAC

Plaintext wewnętrzny:

- Flag (1B) | Dane

Flagi:

- 0x01 - DATA
- 0x02 - END_SESSION

4.5 Stan sesji

Mechanizmy zabezpieczające:

- odrzucanie komunikatów z nieprawidłowym MAC,
- zakończenie sesji przy błędach integralności,
- zamykanie połączenia po odebraniu END_SESSION.

5. Przebieg komunikacji

1. Klient wysyła ClientHello.
2. Serwer odpowiada ServerHello.
3. Obie strony wyprowadzają klucz sesji.
4. Klient wysyła komunikat DATA.
5. Serwer odszyfrowuje i wyświetla treść.
6. Serwer może rozłączyć klienta (kill).
7. Klient może zakończyć sesję (quit → END_SESSION).

Serwer utrzymuje mapę klientów na podstawie adresu ip:port.

6. Dowód działania protokołu Mini-TLS

```
apaliwod@bigubu:~/PSI-Laby/projekt$ docker attach z34_tcp_client
```

```
Unknown command
> connect
Sent ClientHello (p=4294967291, g=2, A=1336062455)
> Received ServerHello (B=3755036500)
*** Secure Connection Established! Key: 884bb4be... ***
send Hello Server! We are working together now! My name is Optimus Prime, we are here, we are waiting...
>
Server ended the session.

Unknown command
> connect
Sent ClientHello (p=4294967291, g=2, A=821524789)
> Received ServerHello (B=2033332998)
*** Secure Connection Established! Key: b37501e1... ***
send Welcome back!
>
Server ended the session.
connect
Sent ClientHello (p=4294967291, g=2, A=404606603)
> Received ServerHello (B=2279115581)
*** Secure Connection Established! Key: 29786b88... ***
send bye...
> quit
Disconnected.
```

```
apaliwod@bigubu:~/PSI-Laby/projekt$ docker attach z34_tcp_server
```

```
> list
Active clients:
>
[+] New connection from 172.21.34.3:41606
[172.21.34.3:41606] Received ClientHello (p=4294967291, g=2, A=1336062455)
[172.21.34.3:41606] Sent ServerHello (B=3755036500)
list
Active clients:
- 172.21.34.3:41606
> [172.21.34.3:41606] Secure Message: Hello Server! We are working together now! My name is Optimus Prime, we are here, we are waiting...
kill 172.21.34.3:41606
Sent EndSession to 172.21.34.3:41606
[-] Connection closed for 172.21.34.3:41606
>
[+] New connection from 172.21.34.3:50662
[172.21.34.3:50662] Received ClientHello (p=4294967291, g=2, A=821524789)
[172.21.34.3:50662] Sent ServerHello (B=2033332998)
[172.21.34.3:50662] Secure Message: Welcome back!
list
Active clients:
- 172.21.34.3:50662
> kill 172.21.34.3:41606
Client not found.
> 172.21.34.3:50662
Unknown command
> kill 172.21.34.3:50662
Sent EndSession to 172.21.34.3:50662
[-] Connection closed for 172.21.34.3:50662
> list
Active clients:
>
[+] New connection from 172.21.34.3:55044
[172.21.34.3:55044] Received ClientHello (p=4294967291, g=2, A=404606603)
[172.21.34.3:55044] Sent ServerHello (B=2279115581)
[172.21.34.3:55044] Secure Message: bye...
[172.21.34.3:55044] Received EndSession request.
[-] Connection closed for 172.21.34.3:55044
list
Active clients:
> exit
```

Kolejność jest zgodna z założeniami protokołu:

1. **ClientHello** – wysyłany przez klienta (parametry DH i A).
2. **ServerHello** – odpowiedź serwera (klucz publiczny B).
3. **SecureMessage (DATA)** – zaszyfrowana wiadomość wraz z HMAC.
4. **EndSession** – wysyłane przez serwer lub klienta.
5. Zamknięcie sesji.

Takie same obserwacje widać dla kolejnych połączeń (porty 50662, 55044). W kolejnych sesjach klucz sesji ma różne wartości (884bb4be..., b37501e1..., 29786b88...). Każde połączenie generuje odmienny klucz sesyjny, mimo stałego **p i g**. Zmienność wynika z losowych kluczy prywatnych używanych w DH.

7. Weryfikacja poprawności szyfrowania

Przy użyciu programu `verify_proof.py`, który przyjmuje klucz sesji oraz szyfrogram w postaci hex, dokonaliśmy weryfikacji poprawności odszyfrowywania.

Po uruchomieniu `verify_proof.py` z kluczem sesyjnym oraz szyfrogramem dostaliśmy taki wynik:

Klucz: a90644cc022afcb0417cebfd0e25b08734800dd62149bec0ee0947644be68460
Szyfrogram: a85225a66c4babd9201884906156d3b606b3

Odszyfrowywanie bajt po bajcie (XOR):

Bajt 0: 0xa8 XOR 0xa9 = 0x1 (' ' jeśli drukowny)
Bajt 1: 0x52 XOR 0x6 = 0x54 ('T' jeśli drukowny)
Bajt 2: 0x25 XOR 0x44 = 0x61 ('a' jeśli drukowny)
Bajt 3: 0xa6 XOR 0xcc = 0x6a ('j' jeśli drukowny)
Bajt 4: 0x6c XOR 0x2 = 0x6e ('n' jeśli drukowny)
Bajt 5: 0x4b XOR 0x2a = 0x61 ('a' jeśli drukowny)
Bajt 6: 0xab XOR 0xfc = 0x57 ('W' jeśli drukowny)
Bajt 7: 0xd9 XOR 0xb0 = 0x69 ('i' jeśli drukowny)
Bajt 8: 0x20 XOR 0x41 = 0x61 ('a' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 4: 0x6c XOR 0x2 = 0x6e ('n' jeśli drukowny)
Bajt 5: 0x4b XOR 0x2a = 0x61 ('a' jeśli drukowny)
Bajt 6: 0xab XOR 0xfc = 0x57 ('W' jeśli drukowny)
Bajt 7: 0xd9 XOR 0xb0 = 0x69 ('i' jeśli drukowny)
Bajt 8: 0x20 XOR 0x41 = 0x61 ('a' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 5: 0x4b XOR 0x2a = 0x61 ('a' jeśli drukowny)
Bajt 6: 0xab XOR 0xfc = 0x57 ('W' jeśli drukowny)
Bajt 7: 0xd9 XOR 0xb0 = 0x69 ('i' jeśli drukowny)
Bajt 8: 0x20 XOR 0x41 = 0x61 ('a' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 6: 0xab XOR 0xfc = 0x57 ('W' jeśli drukowny)
Bajt 7: 0xd9 XOR 0xb0 = 0x69 ('i' jeśli drukowny)
Bajt 8: 0x20 XOR 0x41 = 0x61 ('a' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 6: 0xab XOR 0xfc = 0x57 ('W' jeśli drukowny)
Bajt 7: 0xd9 XOR 0xb0 = 0x69 ('i' jeśli drukowny)
Bajt 8: 0x20 XOR 0x41 = 0x61 ('a' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 10: 0x84 XOR 0xeb = 0x6f ('o' jeśli drukowny)
Bajt 11: 0x90 XOR 0xfd = 0x6d ('m' jeśli drukowny)
Bajt 9: 0x18 XOR 0x7c = 0x64 ('d' jeśli drukowny)
Bajt 10: 0x84 XOR 0xeb = 0x6f ('o' jeśli drukowny)
Bajt 11: 0x90 XOR 0xfd = 0x6d ('m' jeśli drukowny)
Bajt 10: 0x84 XOR 0xeb = 0x6f ('o' jeśli drukowny)
Bajt 11: 0x90 XOR 0xfd = 0x6d ('m' jeśli drukowny)
Bajt 12: 0x61 XOR 0xe = 0x6f ('o' jeśli drukowny)
Bajt 11: 0x90 XOR 0xfd = 0x6d ('m' jeśli drukowny)
Bajt 12: 0x61 XOR 0xe = 0x6f ('o' jeśli drukowny)
Bajt 12: 0x61 XOR 0xe = 0x6f ('o' jeśli drukowny)
Bajt 13: 0x56 XOR 0x25 = 0x73 ('s' jeśli drukowny)
Bajt 13: 0x56 XOR 0x25 = 0x73 ('s' jeśli drukowny)
Bajt 14: 0xd3 XOR 0xb0 = 0x63 ('c' jeśli drukowny)
Bajt 15: 0xb6 XOR 0x87 = 0x31 ('1' jeśli drukowny)
Bajt 16: 0x6 XOR 0x34 = 0x32 ('2' jeśli drukowny)
Bajt 17: 0xb3 XOR 0x80 = 0x33 ('3' jeśli drukowny)

Odszyfrowany HEX: 0154616a6e61576961646f6d6f7363313233

Odszyfrowany TEKST: TajnaWiadomosc123

Treść wiadomości jest zgodna z tym co wysłaliśmy ze strony klienta:

```
*** Secure Connection Established! Key: a90644cc... ***
send TajnaWiadomosc123
[RAPORT] Wysyłanie zaszyfrowanej wiadomosci:
  Ciphertext (Hex): a85225a66c4babd9201884906156d3b606b3
```

8. Wniosek końcowy

Niezależna analiza wykonana w programie **verify_proof.py** potwierdza, że:

1. Klient i serwer ustaliły identyczny klucz sesji Diffie-Hellmana.
2. Szyfrowanie XOR z kluczem sesyjnym działa poprawnie.
3. Wiadomość przesłana w sesji jest możliwa do jednoznacznej rekonstrukcji na podstawie klucza i szyfrogramu.
4. Zaimplementowany protokół **W1 (Encrypt-then-MAC)** zachowuje integralność danych – poprawny MAC umożliwia deszyfrację, błędny zostałby odrzucony.